

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий
Кафедра Информационная безопасность
вычислительных систем и сетей

«Создание GUI-приложения»

Отчет

По лабораторной работе №9
по дисциплине
Скриптовые языки программирования

РУКОВОДИТЕЛЬ:

Вайнбаум Д.А.

(подпись)

СТУДЕНТ:

Браилко В.А.

(подпись)

С22-СИБ

Работа защищена «__» _____

С оценкой _____

Нижний Новгород 2026

Цель: освоить принципы разработки десктопных приложений с графическим интерфейсом на Python, включая работу с визуальными компонентами и событийно-ориентированную модель взаимодействия с пользователем.

Постановка задачи: создать на Python настольное приложение с использованием выбранной GUI-библиотеки, содержащее поле ввода, кнопку и текстовую метку, где нажатие кнопки вызывает обработчик события для получения пользовательских данных и вывода результата непосредственно в интерфейс.

Реализация

Для обеспечения стабильности работы и предотвращения блокировки интерфейса при выполнении вычислительных операций, бизнес-логика была полностью отделена от визуального слоя. Подобное разделение ответственности (Separation of Concerns) позволяет изолировать криптографические алгоритмы генерации и анализа стойкости паролей от механизмов отрисовки виджетов и обработки событий GUI-потока. В качестве вычислительного ядра был реализован модуль расчёта энтропии, генерации криптографически стойких последовательностей и оценки времени перебора. Интеграция данного модуля демонстрирует принцип повторного использования кода и модульности: вычислительные операции инкапсулированы в независимые классы, которые могут вызываться из любого интерфейса без модификации внутренней реализации. При этом интерфейс не содержит математической логики, а лишь транслирует пользовательский ввод в контроллер и отображает результат.

Архитектура решения разделена на четыре функциональных модуля:

- `core.py` – ядро вычислений и валидации. Содержит класс `DataValidator` для проверки параметров, `PasswordGenerator` на базе модуля `secrets` для создания паролей и `PasswordAnalyzer` для расчёта энтропии и классификации стойкости.
- `controller.py` – связующее звено (Mediator). Принимает параметры конфигурации и сырой ввод из интерфейса, делегирует проверку валидатору, вызывает соответствующий метод вычислительного ядра и возвращает отформатированную строку для отображения.
- `gui.py` – визуальный слой. Наследует `tk.Tk`, управляет созданием и компоновкой виджетов (`pack`, `grid`), регистрирует обработчики событий (`command`, `bind`) и обновляет поле вывода без блокировки основного потока.
- `main.py` – точка входа. Инициализирует главное окно приложения и запускает бесконечный цикл обработки событий.

Файл core.py:

```
class DataValidator:
    @staticmethod
    def validate(length: int, u: bool, l: bool, d: bool, s: bool)
    -> tuple[bool, str]:
        """Проверка входных данных на корректность"""
        if length <= 0:
            return False, "Длина пароля должна быть больше нуля"
        if not (u or l or d or s):
            return False, "Выберите хотя бы один набор символов"
        return True, ""

class PasswordAnalyzer:
    @staticmethod
    def get_pool_size(u: bool, l: bool, d: bool, s: bool) -> int:
        """Мощность алфавита"""
        size = 0
        if u: size += 26
        if l: size += 26
        if d: size += 10
        if s: size += 32 # стандартные спецсимволы
        return size

    @staticmethod
    def calculate_entropy(length: int, pool_size: int) -> float:
        """Расчёт битовой энтропии:  $E = L * \log_2(N)$ ."""
        if pool_size <= 1: return 0.0
        return length * math.log2(pool_size)

    @staticmethod
    def estimate_time(pool_size: int, length: int, speed: float =
1e10) -> str:
        """Оценка времени полного перебора (GPU ~10^10
attempts/sec)."""
```

```

combinations = pool_size ** length
seconds = combinations / speed
if seconds < 1: return "< 1 секунды"
elif seconds < 60: return f"{seconds:.1f} сек."
elif seconds < 3600: return f"{seconds/60:.1f} мин."
elif seconds < 86400: return f"{seconds/3600:.1f} ч."
elif seconds < 31536000: return f"{seconds/86400:.1f} дн."
return f"{seconds/31536000:.1f} лет"

```

```

@staticmethod

```

```

def classify_strength(entropy: float) -> str:

```

```

    """Классификация стойкости по шкале энтропии
(NIST/OWASP)."""

```

```

    if entropy < 28: return "Критически слабый"
    elif entropy < 36: return "Слабый"
    elif entropy < 59: return "Средний"
    elif entropy < 80: return "Сильный"
    return "Очень сильный"

```

```

class PasswordGenerator:

```

```

    @staticmethod

```

```

    def generate(length: int, u: bool, l: bool, d: bool, s: bool)
-> str:

```

```

    """Криптографически стойкая генерация пароля"""

```

```

    pool = ""

```

```

    if u: pool += string.ascii_uppercase

```

```

    if l: pool += string.ascii_lowercase

```

```

    if d: pool += string.digits

```

```

    if s: pool += string.punctuation

```

```

    if not pool: return ""

```

```

    return ''.join(secrets.choice(pool) for _ in
range(length))

```

Файл controller.py:

```
class AppController:
```

```
    def __init__(self):
```

```
        self.validator = DataValidator()
```

```
        self.analyzer = PasswordAnalyzer()
```

```
        self.generator = PasswordGenerator()
```

```
    def generate(self, length: int, u: bool, l: bool, d: bool, s: bool) -> str:
```

```
        """Маршрутизация запроса на генерацию пароля."""
```

```
        is_valid, err = self.validator.validate(length, u, l, d, s)
```

```
        if not is_valid: return err
```

```
        pwd = self.generator.generate(length, u, l, d, s)
```

```
        pool = self.analyzer.get_pool_size(u, l, d, s)
```

```
        entropy = self.analyzer.calculate_entropy(length, pool)
```

```
        strength = self.analyzer.classify_strength(entropy)
```

```
        time_est = self.analyzer.estimate_time(pool, length)
```

```
        return (f"Сгенерированный пароль: {pwd}\n"
```

```
                f"Энтропия: {entropy:.2f} бит\n"
```

```
                f"Стойкость: {strength}\n"
```

```
                f"Время подбора: {time_est}")
```

```
    def analyze(self, raw_pwd: str) -> str:
```

```
        """Маршрутизация запроса на анализ
пользовательского пароля."""
```

```
        if not raw_pwd.strip(): return "Введите пароль
для анализа"
```

```
        # Автоматическое определение используемых наборов
СИМВОЛОВ
```

```
        has_u = any(c.isupper() for c in raw_pwd)
        has_l = any(c.islower() for c in raw_pwd)
        has_d = any(c.isdigit() for c in raw_pwd)
        has_s = any(c in string.punctuation for c in
raw_pwd)

        length = len(raw_pwd)
        pool = self.analyzer.get_pool_size(has_u, has_l,
has_d, has_s)
        entropy = self.analyzer.calculate_entropy(length,
pool)

        strength =
self.analyzer.classify_strength(entropy)
        time_est = self.analyzer.estimate_time(pool,
length)

        return (f"Анализ пароля: {raw_pwd}\n"
                f"Длина: {length} символов\n"
                f"Энтропия: {entropy:.2f} бит\n"
                f"Стойкость: {strength}\n"
                f"Время подбора: {time_est}")
```

Файл gui.py:

```
class MainWindow(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Анализатор и генератор паролей")
        self.geometry("580x520")
        self.resizable(False, False)

        self.controller = AppController()
        self.length_var = tk.IntVar(value=16)
        self.vars = {
            "upper": tk.BooleanVar(value=True),
            "lower": tk.BooleanVar(value=True),
            "digits": tk.BooleanVar(value=True),
            "symbols": tk.BooleanVar(value=False)
        }
        self._build_ui()

    def _build_ui(self):
        tk.Label(self, text="Параметры конфигурации:",
font=("Segoe UI", 12, "bold")).pack(pady=(15, 5))

        params_frame = tk.Frame(self)
        params_frame.pack(pady=5)

        tk.Label(params_frame, text="Длина:", font=("Segoe UI",
11)).grid(row=0, column=0, padx=10, sticky="e")

        ttk.Spinbox(params_frame, from_=1, to=128,
textvariable=self.length_var, width=8).grid(row=0, column=1,
pady=5)

        opts_frame = tk.Frame(self)
        opts_frame.pack(pady=10)
```



```
tk.Label(opts_frame, text="Наборы символов:", font=("Segoe UI", 11)).pack(anchor="w")
```

```
chk_frame = tk.Frame(opts_frame)
```

```
chk_frame.pack()
```

```
ttk.Checkbutton(chk_frame, text="A-Z",  
variable=self.vars["upper"]).pack(side=tk.LEFT, padx=10)
```

```
ttk.Checkbutton(chk_frame, text="a-z",  
variable=self.vars["lower"]).pack(side=tk.LEFT, padx=10)
```

```
ttk.Checkbutton(chk_frame, text="0-9",  
variable=self.vars["digits"]).pack(side=tk.LEFT, padx=10)
```

```
ttk.Checkbutton(chk_frame, text="!@#",  
variable=self.vars["symbols"]).pack(side=tk.LEFT, padx=10)
```

```
btn_frame = tk.Frame(self)
```

```
btn_frame.pack(pady=15)
```

```
tk.Button(btn_frame, text="Сгенерировать", font=("Segoe UI", 11, "bold"),
```

```
bg="#2196F3", fg="white",  
command=self._on_generate).pack(side=tk.LEFT, padx=10)
```

```
tk.Button(btn_frame, text="Анализировать введённый",  
font=("Segoe UI", 11, "bold"),
```

```
bg="#4CAF50", fg="white",  
command=self._on_analyze).pack(side=tk.LEFT, padx=10)
```

```
tk.Label(self, text="Введите пароль для анализа  
(опционально):", font=("Segoe UI", 11)).pack(pady=(5, 2))
```

```
self.manual_input = tk.Entry(self, font=("Consolas", 11),  
width=50)
```

```
self.manual_input.pack(pady=5)
```

```
tk.Label(self, text="Результат:", font=("Segoe UI",  
11)).pack(pady=(5, 2))
```

```

        self.output_text = tk.Text(self, font=("Consolas", 11),
height=9, width=65,

                                state="disabled", bg="#f4f6f8")

        self.output_text.pack(pady=5)


        self.manual_input.bind("<Return>", lambda evt:
self._on_analyze())


def _on_generate(self):
    """Обработчик кнопки генерации."""
    length = self.length_var.get()
    u, l, d, s = self.vars["upper"].get(),
self.vars["lower"].get(), self.vars["digits"].get(),
self.vars["symbols"].get()
    result = self.controller.generate(length, u, l, d, s)
    self._update_output(result)


def _on_analyze(self):
    """Обработчик кнопки анализа или нажатия Enter."""
    pwd = self.manual_input.get().strip()
    result = self.controller.analyze(pwd)
    self._update_output(result)


def _update_output(self, text: str):
    """Безопасное обновление текстового поля вывода без
блокировки GUI."""
    self.output_text.config(state="normal")
    self.output_text.delete("1.0", tk.END)
    self.output_text.insert("1.0", text)
    self.output_text.config(state="disabled")

```

Файл main.py:

```
if __name__ == "__main__":  
    app = MainWindow()  
    app.mainloop() # Запуск бесконечного цикла ожидания событий  
GUI
```

Результаты тестирования

В ходе тестирования приложения была проверена корректность работы всех реализованных функций графического интерфейса и вычислительного ядра. Тестирование проводилось путём последовательного выполнения типовых операций в рабочем окружении debian 12 (Python 3.11, стандартная библиотека tkinter и модули secrets, math, string). Проверялись следующие аспекты: корректность генерации, точность расчёта энтропии, классификация стойкости, оценка времени подбора, а также реакция интерфейса на некорректный, пустой ввод.

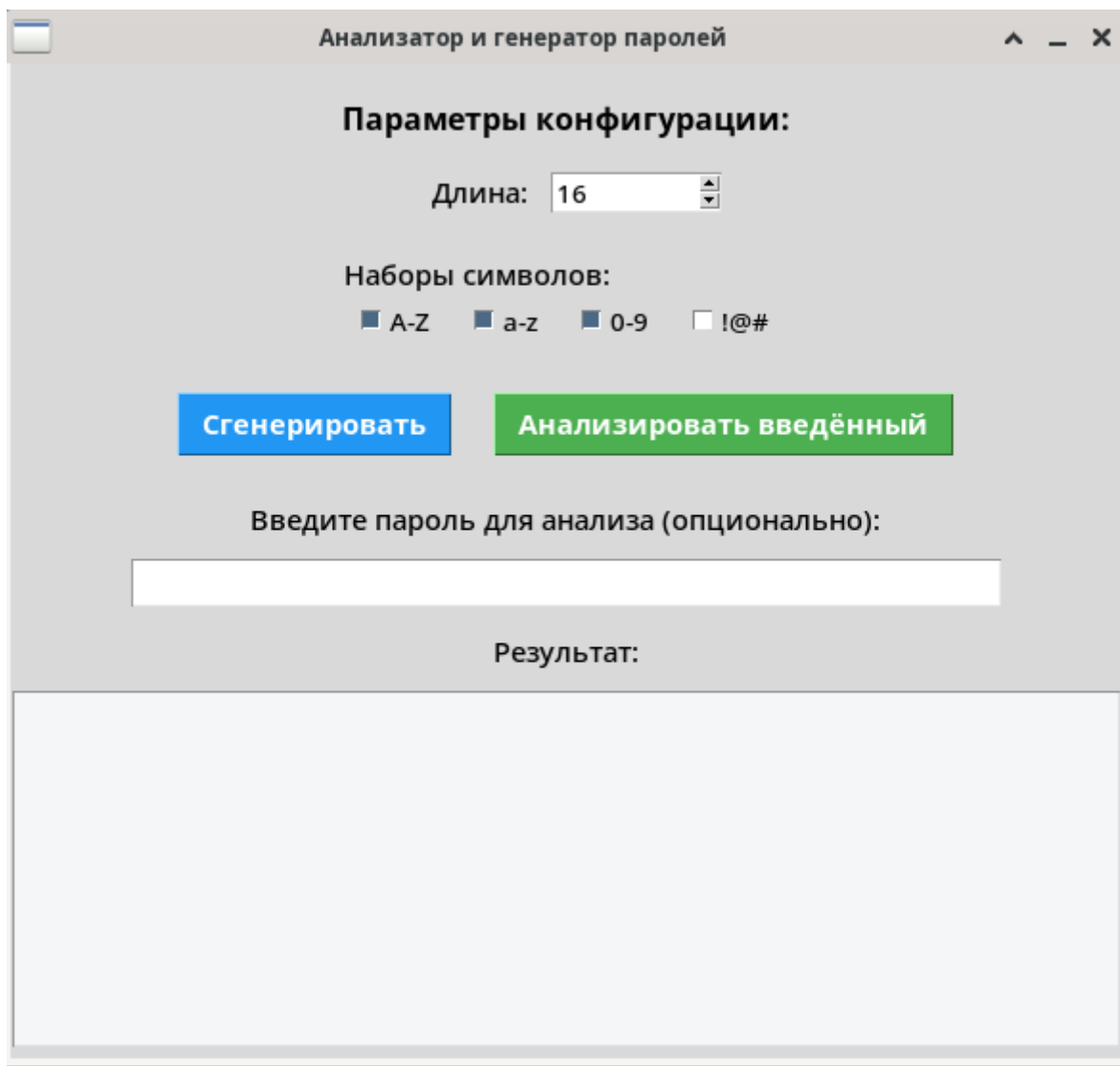


Рис 1.

Отображается пустое окно приложения с размещёнными элементами: настройка длины пароля, чекбоксы выбора наборов символов, две функциональные кнопки, поле ручного ввода и область вывода результата. Интерфейс загружен без ошибок, элементы выровнены согласно заданной компоновке, фокус установлен на поле ручного ввода.

Анализатор и генератор паролей

Параметры конфигурации:

Длина: 20

Наборы символов:

☒ A-Z ☒ a-z ☒ 0-9 ☐ !@#

Сгенерировать **Анализировать введённый**

Введите пароль для анализа (опционально):

Результат:

🔑 Сгенерированный пароль: AP7GjYsflhCJLmmkBTso
Энтропия: 119.08 бит
🛡️ Стойкость: Очень сильный
🕒 Время подбора: 2233712029258619136.0 лет

Рис 2.

Выбрана длина 20, активированы все наборы символов. После нажатия кнопки «Сгенерировать» в поле результата отображается случайная последовательность, категория Очень сильный и оценка времени подбора $> 10^{15}$ лет. Подтверждает корректность работы модуля secrets и формулы расчёта энтропии.

Анализатор и генератор паролей

Параметры конфигурации:

Длина: 20

Наборы символов:

☒ A-Z ☒ a-z ☒ 0-9 ☐ !@#

Сгенерировать **Анализировать введённый**

Введите пароль для анализа (опционально):

MyPass123

Результат:

Анализ пароля: MyPass123

✎ Длина: 9 символов

📊 Энтропия: 53.59 бит

🛡 Стойкость: Средний

⌚ Время подбора: 15.7 дн.

Рис 3.

В поле ручного ввода введена строка MyPass123, нажата кнопка «Анализировать введённый» или клавиша Enter. Результат выполнения автоматически определяет использованные наборы символов, рассчитывает энтропию 53.59 бит и присваивает категорию Средний. Подтверждает корректность обратной эвристики и изоляцию аналитической логики.

Рис 4.

При снятии всех чекбоксов и нажатии кнопки генерации срабатывает встроенный валидатор. В поле вывода отображается сообщение «Выберите хотя бы один набор символов». Исключительные ситуации не генерируются, интерфейс остаётся полностью отзывчивым.

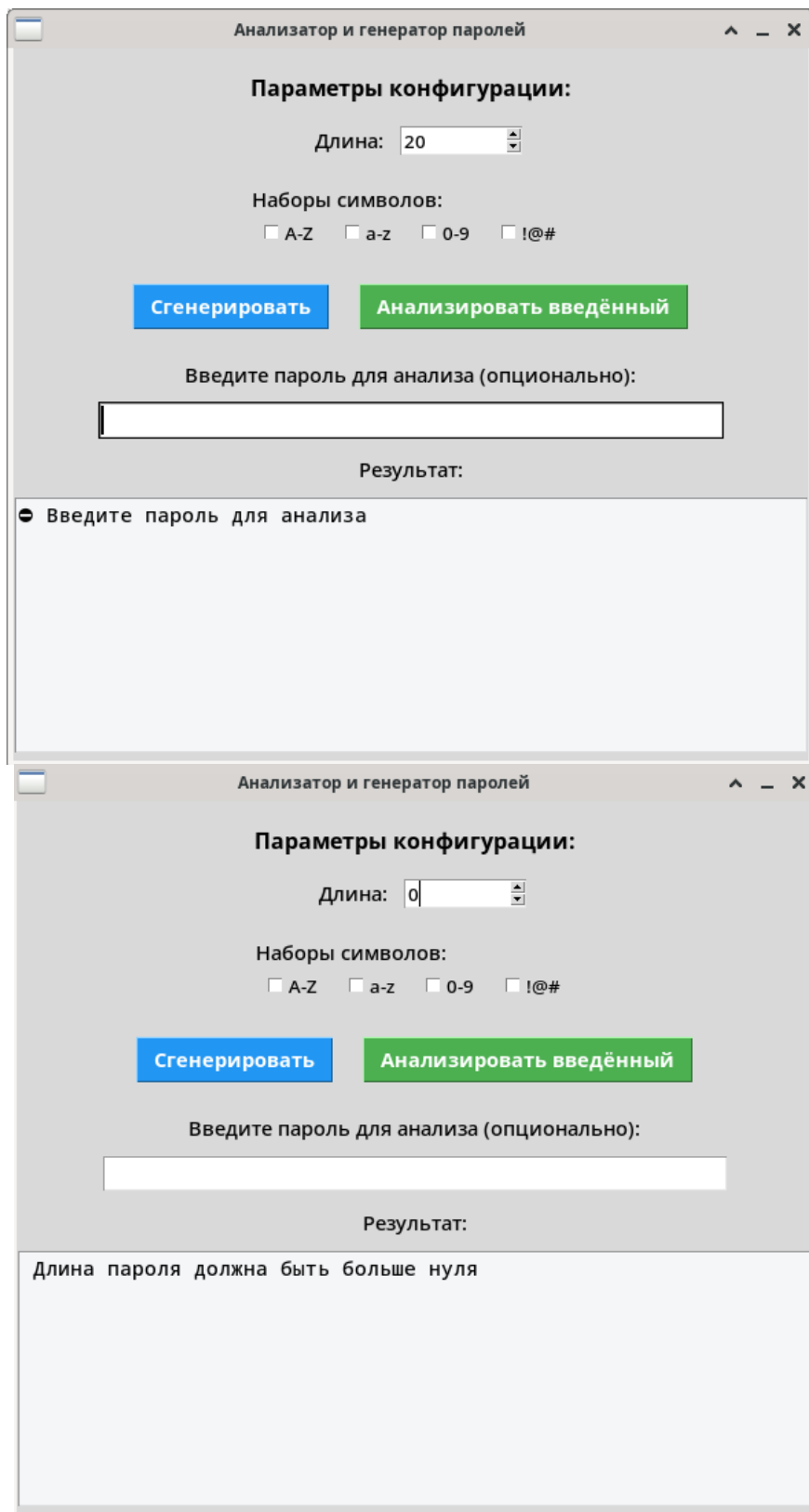


Рис 5.

При попытке проанализировать пустую строку или задать длину пароля 0 валидатор блокирует вычислительную операцию, выводя соответствующее предупреждение. Подтверждает устойчивость алгоритма к нештатным данным и корректную работу механизма защиты от переполнения/нелогичных вводов.

Вывод

В результате работы закреплены навыки проектирования оконных приложений на базе стандартной библиотеки `tkinter`, управления компоновкой виджетов, регистрации `callback`-обработчиков и организации безопасного межмодульного взаимодействия. Полученный опыт формирует прочную основу для разработки сложных десктопных решений в области информационной безопасности, автоматизации рутинных процессов и создания пользовательских инструментов с интуитивным графическим интерфейсом.

Практическая реализация подтвердила эффективность модульной структуры приложения, состоящей из изолированных компонентов вычислительного ядра, валидатора, контроллера и графического интерфейса. Интеграция алгоритма расчёта информационной энтропии и стойкой генерации продемонстрировала принципы повторного использования кода и масштабируемости системы.